Machine Translated by Google



C++ - Módulo 05

Repetição e Exceções

Resumo:
Este documento contém os exercícios do Módulo 05 dos módulos C++.

Versão: 10.2

Machine Translated by Google

Conteúdo

EU Introdução 2

II Regras gerais 3

III Exercício 00: Mamãe, quando eu crescer, quero ser burocrata! 6

IV Exercício 01: Formem-se, vermes! 8

V Exercício 02: Não, você precisa do formulário 28B, não do 28C... 10

VI Exercício 03: Pelo menos isso é melhor do que fazer café 12

VII Submissão e Avaliação por Pares 14

Machine Translated by Google

Capítulo I

Introdução

C++ é uma linguagem de programação de propósito geral criada por Bjarne Stroustrup como uma extensão da linguagem de programação C, ou "C com Classes" (fonte: Wikipedia).

O objetivo destes módulos é apresentar a **Programação Orientada a Objetos.**
Este será o ponto de partida da sua jornada C++. Muitas linguagens são recomendadas para aprender POO. Escolhemos C++ porque é derivado do seu velho amigo, C.
Como esta é uma linguagem complexa e para manter as coisas simples, seu código estará em conformidade com o padrão C++98.

Estamos cientes de que o C++ moderno é significativamente diferente em muitos aspectos. Então, se você quer se tornar um desenvolvedor C++ proficiente, cabe a você ir além depois do 42 Common Core!

Machine Translated by Google

Capítulo II

Regras gerais

Compilando

- Compile seu código com c++ e os sinalizadores -Wall -Wextra -Werror
- Seu código ainda deve compilar se você adicionar o sinalizador -std=c++98

Convenções de formatação e nomenclatura

- Os diretórios dos exercícios serão nomeados desta forma: ex00, ex01, ... , exn
- Nomeie seus arquivos, classes, funções, funções de membro e atributos conforme necessário em as diretrizes.
- Escreva nomes de classes no formato **UpperCamelCase .** Arquivos contendo código de classe serão sempre será nomeado de acordo com o nome da classe. Por exemplo: ClassName.hpp/ClassName.h, ClassName.cpp ou ClassName.tpp. Então, se você tiver um arquivo de cabeçalho contendo a definição de uma classe "BrickWall" representando uma parede de tijolos, seu nome será BrickWall.hpp.
- A menos que especificado de outra forma, todas as mensagens de saída devem ser encerradas por uma nova linha caractere e exibido na saída padrão.
- *Adeus Norminette!* Nenhum estilo de codificação é imposto nos módulos C++. Você pode seguir seu favorito. Mas tenha em mente que um código que seus avaliadores pares não conseguem entender é um código que eles não conseguem classificar. Faça o seu melhor para escrever um código limpo e legível.

Permitido/Proibido

Você não está mais codificando em C. Hora de C++! Portanto:

- Você tem permissão para usar quase tudo da biblioteca padrão. Assim, em vez de ficar preso ao que você já sabe, seria inteligente usar o máximo possível as versões C++-ish das funções C com as quais você está acostumado.
- No entanto, você não pode usar nenhuma outra biblioteca externa. Isso significa que as bibliotecas C++11 (e formas derivadas) e Boost são proibidas. As seguintes funções também são proibidas: *printf(), *alloc() e free(). Se você usá-las, sua nota será 0 e pronto.

Machine Translated by Google

- Observe que, a menos que explicitamente indicado de outra forma, o uso do namespace <ns_name> e palavras-chave de amigos são proibidas. Caso contrário, sua nota será -42.
- **Você tem permissão para usar o STL somente no Módulo 08 e 09.** Isso significa: nenhum **Container** (vetor/lista/mapa/e assim por diante) e nenhum **Algoritmo** (qualquer coisa que exija incluir o cabeçalho <algoritmo>) até então. Caso contrário, sua nota será -42.

Alguns requisitos de design

- Vazamento de memória ocorre em C++ também. Quando você aloca memória (usando o novo palavra-chave), você deve evitar **vazamentos de memória.**
- Do Módulo 02 ao Módulo 09, suas aulas devem ser elaboradas na Escola **Ortodoxa Forma canônica, exceto quando explicitamente indicado de outra forma.**
- Qualquer implementação de função colocada em um arquivo de cabeçalho (exceto para modelos de função) significa 0 para o exercício.
- Você deve ser capaz de usar cada um dos seus cabeçalhos independentemente dos outros. Assim, eles devem incluir todas as dependências de que precisam. No entanto, você deve evitar o problema de inclusão dupla adicionando **guardas de inclusão.** Caso contrário, sua nota será 0.

Leia-me

- Você pode adicionar alguns arquivos adicionais se precisar (por exemplo, para dividir seu código). Como essas atribuições não são verificadas por um programa, sinta-se à vontade para fazê-lo, desde que entregue os arquivos obrigatórios.
- Às vezes, as diretrizes de um exercício parecem curtas, mas os exemplos podem mostrar requisitos que não estão explicitamente escritos nas instruções.
- Leia cada módulo completamente antes de começar! Sério, faça.
- Por Odin, por Thor! Use seu cérebro!!!



Em relação ao Makefile para projetos C++, as mesmas regras do C se aplicam (veja o capítulo Norma sobre o Makefile).



Você terá que implementar muitas classes. Isso pode parecer tedioso, a menos que você consiga criar scripts para seu editor de texto favorito.

Machine Translated by Google



Você tem uma certa liberdade para completar os exercícios.
No entanto, siga as regras obrigatórias e não seja preguiçoso. Você iria
perca muitas informações úteis! Não hesite em ler sobre
conceitos teóricos.

Machine Translated by Google

Capítulo III

Exercício 00: Mamãe, quando eu crescer, quero ser burocrata!

	Exercício: 00
Mamãe, quando eu crescer, quero ser burocrata!	
Diretório de entrega: ex00/	
Arquivos para entrega: Makefile, main.cpp, Bureaucrat.{h, hpp}, Bureaucrat.cpp Funções proibidas: Nenhuma	



Observe que classes de exceção não precisam ser projetadas na Orthodox Canonical Form. No entanto, todas as outras classes devem segui-la.

Vamos projetar um pesadelo artificial de escritórios, corredores, formulários e filas de espera. Parece divertido? Não? Que pena.

Primeiro, comece com a menor engrenagem dessa vasta máquina burocrática: o **Burocrata.**

Um **burocrata** deve ter:

• Um nome constante.

• Uma nota que varia de **1** (nota mais alta possível) a **150** (nota mais baixa possível).

Qualquer tentativa de instanciar um Burocrata com uma nota inválida deve lançar uma exceção: uma Bureaucrat::GradeTooHighException ou uma Bureaucrat::GradeTooLowException.

Machine Translated by Google

Você fornecerá getters para ambos os atributos: getName() e getGrade(). Você também deve implementar duas funções de membro para incrementar ou decrementar a nota do burocrata. Se a nota sair do intervalo, ambas as funções devem lançar as mesmas exceções que a construtor.



Lembre-se, como o grau 1 é o mais alto e 150 o mais baixo, aumentar o grau 3 deve resultar em um grau 2 para o burocrata.

As exceções lançadas devem ser capturáveis usando blocos try e catch:

```
tentar
{
        /* fazer algumas coisas com burocratas */
}
catch (std::exception & e) {

        /* manipular exceção */
}
```

Você deve implementar uma sobrecarga do operador de inserção («) para imprimir a saída no seguinte formato (sem os colchetes angulares):

<nome>, grau burocrático <grau>.

Como de costume, envie alguns testes para provar que tudo funciona conforme o esperado.

Machine Translated by Google

Capítulo IV

Exercício 01: Formem-se, vermes!

	Exercício: 01
Forme uma fila, vermes!	
Diretório de entrega: ex01/	
Arquivos para entregar: Arquivos do exercício anterior + Form.{h, hpp}, Form.cpp Funções proibidas: Nenhuma	

Agora que você tem burocratas, vamos dar a eles algo para fazer. Que atividade melhor poderia haver do que preencher uma pilha de formulários?

Vamos criar uma classe **Form .** Ela tem:

• Um nome constante.

• Um booleano indicando se está assinado (na construção, não está).

• Uma nota constante é necessária para assiná-lo.

• Uma inclinação constante necessária para executá-lo.

Todos esses atributos são **privados,** não protegidos.

As notas do **Formulário** seguem as mesmas regras do Burocrata. Assim, o
As seguintes exceções serão geradas se a nota de um formulário estiver fora dos limites:
Form::GradeTooHighException e Form::GradeTooLowException.

Como antes, escreva getters para todos os atributos e sobrecarregue o operador de inserção («) para imprimir todas as informações do formulário.

Machine Translated by Google

Além disso, adicione uma função membro beSigned() ao Form que recebe um Burocrata como parâmetro. Ela altera o status do formulário para assinado se a nota do burocrata for alta o suficiente (maior ou igual à necessária). Lembre-se, a nota 1 é maior que a nota 2.
Se a nota for muito baixa, lance uma Form::GradeTooLowException.

Então, modifique a função membro signForm() na classe Bureaucrat. Esta função deve chamar Form::beSigned() para tentar assinar o formulário. Se o formulário for assinado com sucesso, ele imprimirá algo como: <bureaucrat> signed
<form>

Caso contrário, ele imprimirá algo como:

<burocrata> não conseguiu assinar <formulário> por <motivo>.

Implemente e envie alguns testes para garantir que tudo funcione conforme o esperado.

Machine Translated by Google

Capítulo V

Exercício 02: Não, você precisa do formulário 28B, não do 28C...

	Exercício: 02
Não, você precisa do formulário 28B, não do 28C...	
Diretório de entrega: ex02/	
Arquivos para entregar: Makefile, main.cpp, Bureaucrat.[{h, hpp},cpp], Bureaucrat.cpp + AForm.[{h, hpp},cpp], ShrubberyCreationForm.[{h, hpp},cpp], + RobotomyRequestForm.[{h, hpp},cpp], PresidentialPardonForm.[{h, hpp},cpp]	
Funções proibidas: Nenhuma	

Agora que você tem formulários básicos, é hora de criar mais alguns que realmente façam alguma coisa.

Em todos os casos, a classe base Form deve ser uma classe abstrata e, portanto, deve ser renomeada para AForm. Tenha em mente que os atributos do formulário precisam permanecer privados e que eles pertencem à classe base.

Adicione as seguintes classes concretas:

- **ShrubberyCreationForm:** Notas necessárias: sinal 145, exec 137
  Cria um arquivo <target>_shrubbery no diretório de trabalho e grava árvores ASCII dentro dele.

- **RobotomyRequestForm:** Notas necessárias: sign 72, exec 45 Faz alguns ruídos de perfuração e informa que <target> foi robotomizado com sucesso 50% das vezes. Caso contrário, informa que a robotomia falhou.

- **PresidentialPardonForm:** Notas necessárias: sinal 25, exec 5
  Informa que <alvo> foi perdoado por Zaphod Beeblebrox.

Todos eles levam apenas um parâmetro em seu construtor: o alvo do formulário. Para por exemplo, "casa" se você quiser plantar arbustos em casa.

Machine Translated by Google

Agora, adicione a função membro execute(Bureaucrat const & executor) const ao formulário base e implemente uma função para executar a ação do formulário nas classes concretas. Você deve verificar se o formulário está assinado e se o grau do burocrata que tenta executar o formulário é alto o suficiente. Caso contrário, lance uma exceção apropriada.

Se você verifica os requisitos em cada classe concreta ou na classe base (e então chama outra função para executar o formulário) é com você. No entanto, uma maneira é mais elegante do que a outra.

Por fim, adicione a função membro executeForm(AForm const & form) ao Bureau-
classe crat. Ela deve tentar executar o formulário. Se bem-sucedido, imprima algo como:

<burocrata> executou <formulário>

Caso contrário, imprima uma mensagem de erro explícita.

Implemente e envie alguns testes para garantir que tudo funcione conforme o esperado.

Machine Translated by Google

Capítulo VI

Exercício 03: Pelo menos isso é melhor do que fazer café

	Exercício: 03
Pelo menos isso é melhor do que fazer café	
Diretório de entrega: ex03/	
Arquivos para entregar: Arquivos de exercícios anteriores + Intern.{h, hpp}, Intern.cpp Funções proibidas: Nenhuma	

Como preencher formulários o dia todo seria muito cruel para nossos burocratas, os estagiários existem para assumir essa tarefa tediosa. Neste exercício, você deve implementar a classe **Intern .** O estagiário não tem nome, nem nota, nem características únicas. A única coisa com que os burocratas se importam é que eles façam seu trabalho.

No entanto, o estagiário tem uma habilidade essencial: a função makeForm(). Essa função recebe duas strings como parâmetros: a primeira representa o nome de um formulário, e a segunda representa o alvo do formulário. Ela retorna um ponteiro para um objeto **Form** (correspondente ao nome do formulário passado como parâmetro), com seu alvo inicializado para o segundo parâmetro.

Deve imprimir algo como:

Estagiário cria <formulário>

Se o nome do formulário fornecido não existir, imprima uma mensagem de erro explícita.

Machine Translated by Google

Você deve evitar soluções ilegíveis e confusas, como usar uma estrutura if/el-seif/else excessiva. Esse tipo de abordagem não será aceito durante o processo de avaliação. Você não está mais no Piscine (pool). Como de costume, você deve testar tudo para garantir que funcione conforme o esperado.

Por exemplo, o código a seguir cria um **RobotomyRequestForm** direcionado a "Dobrador":

```
{
        Interno algumRandomInternal;
        Formulário* rrf;

        rrf = someRandomIntern.makeForm(" solicitação de robotomia", "Bender");
}
```

Machine Translated by Google

Capítulo VII

Submissão e Avaliação por Pares

Envie sua tarefa para seu repositório Git como de costume. Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Certifique-se de verificar novamente os nomes de suas pastas e arquivos para garantir que estejam corretos.



16D85ACC441674FBA2DF65190663F9373230CEAB1E4A0818611C0E39F5B26E4D774F1
74620A16827E1B16612137E59ECD492E468A92DCB17BF16988114B98587594D12810
E67D173222A